

Faculdade de Engenharia da Universidade do Porto



Escape The Lab

Rafael Magalhães up202206499

Ricardo Oliveira up202205353

Vitor Pereira up202207715

1. Instruções de Utilização

1.1. Ecrã Inicial



Figura 1. Ecrã inicial da aplicação

A aplicação inicia com o ecrã do menu principal que apresenta um cursor e duas opções que permitem jogar e sair da aplicação respetivamente. Estas opções são seleccionáveis arrastando o cursor e premindo o botão esquerdo do rato.

1.2. Ecrã de seleção de jogador

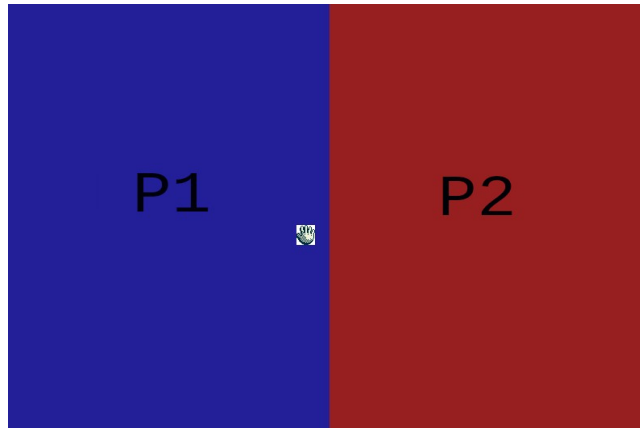


Figura 2. Ecrã de seleção de jogador

O ecrã de seleção de jogador apresenta as opções de jogador 1 ou jogador 2 que podem ser seleccionadas da mesma forma do ecrã inicial. Após o jogador ser seleccionado, o jogo começa.

1.3. Ecrã de jogo

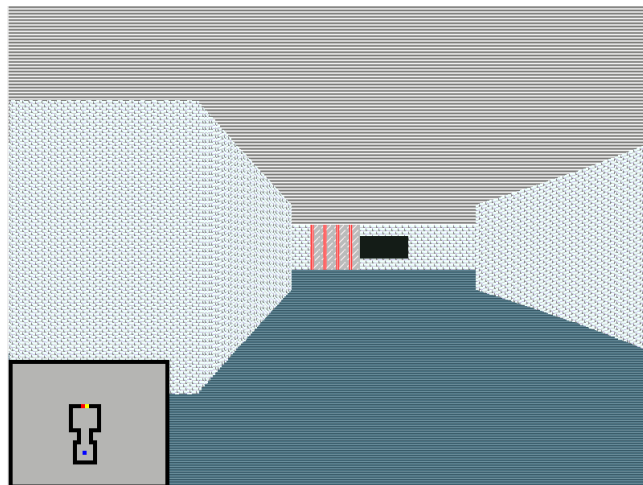


Figura 3. Ecrã principal do jogo

O ecrã principal do jogo apresenta a visão do jogador, assim como um mapa no canto inferior esquerdo. Os controlos do jogador são apresentados na seguinte tabela:

Movimento	Comando
Premir a tecla W	Andar para a frente
Premir a tecla A	Andar para a esquerda
Premir a tecla S	Andar para trás
Premir a tecla D	Andar para a direita
Mover o rato para a esquerda	Rodar no sentido anti-horário
Mover o rato para a direita	Rodar no sentido horário
Premir a tecla ESC	Pausar o jogo
Premir o botão esquerdo do rato	Interagir com os terminais

Figura 4. Tabela de comandos do jogo

Para progredir pelo jogo, o jogador precisa de completar as tarefas pedidas pelos terminais, sob um limite de tempo.

Segue-se uma breve explicação dos níveis:

1. Tutorial: Ativar o terminal.
2. Chegar a todos os terminais num limite temporal.
3. Os dois jogadores têm que introduzir a palavra-passe para escapar.

1.4. Ecrã de pausa

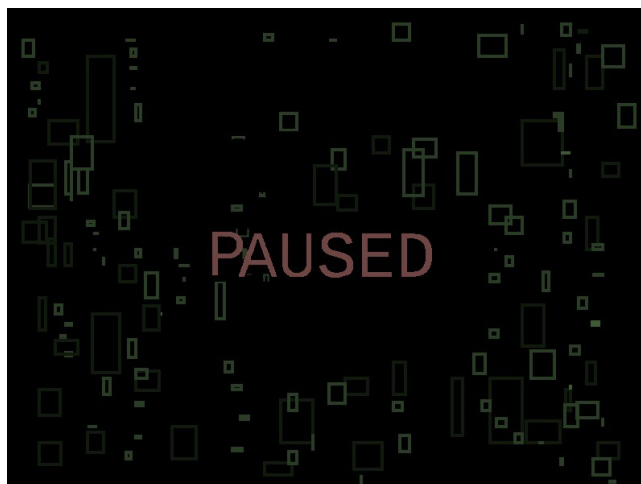


Figura 5. Ecrã de pausa

O ecrã de pausa é controlado de forma idêntica ao menu principal e também apresenta duas opções que permitem voltar ao jogo ou voltar ao menu principal respetivamente.

1.5. Ecrã de vitória

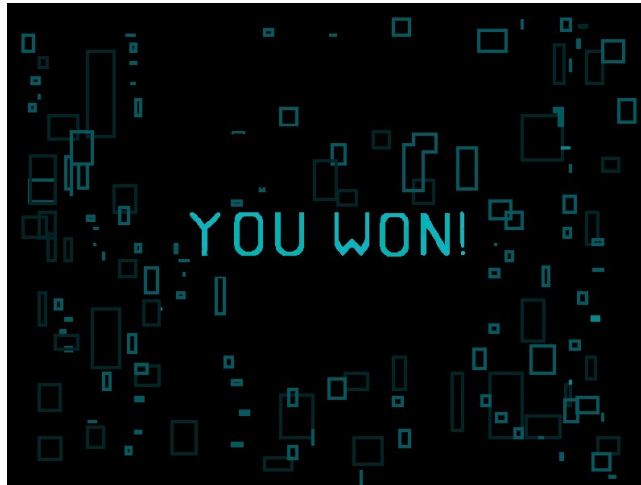


Figura 6. Ecrã de vitória

O ecrã de vitória é muito semelhante ao ecrã de pausa porém apresenta uma pequena mensagem avisando que o jogador venceu e as opções dadas permitem jogar outra vez ou voltar ao menu principal respetivamente.

1.6. Ecrã de derrota



Figura 7. Ecrã de derrota

O ecrã de derrota é funcionalmente idêntico ao ecrã de vitória (permite jogar outra vez ou regressar ao menu), no entanto, mostra uma mensagem que o jogador perdeu.

2. Estado do Projeto

2.1. Vista geral das funcionalidades

As funcionalidades gerais do projeto, assim como alguns detalhes sobre a sua implementação são apresentadas na seguinte tabela:

Funcionalidade	Dispositivo(s) utilizado(s)	Método de utilização
Controlo do <i>frame rate</i>	Temporizador	Interrupções
Navegação nos menus	Rato	Interrupções
Controlo do jogador	Teclado e rato	Interrupções
Visualização dos ecrãs	Placa de vídeo	-
Tempo limite de jogo	RTC	Interrupções
Comunicação entre jogadores	Porta-Série	Polling
Introdução de texto	Teclado	Interrupções

Figura 8. Tabela de funcionalidades

2.2. Funcionalidades do temporizador

O temporizador é utilizado para gerar interrupções periódicas que provocam o redesenhar da tela do projeto. Estas interrupções são captadas pelo loop de driver_receive das aulas, que está na função main_event_loop do ficheiro driver_handler.c. Após a captação de uma interrupção é chamado o handler da mesma (timer_ih no ficheiro timer.c) que incrementa um contador. Quando esse contador chega ao frame rate definido, redesenha a tela chamando a função state_draw_frame no ficheiro game_state.c.

2.3. Funcionalidades do teclado

O teclado é usado para controlar o jogador e para introdução de texto. As suas interrupções são processadas na função main_event_loop tal como o timer e chamam o seu handler, a função kbc_ih no ficheiro kbc.c. Esta função processa o scancode recebido e envia-o para a função state_kbd_event no ficheiro game_state.c que processa os efeitos do premir da tecla no jogo. No caso do movimento do jogador o scancode é enviado para a função player_move no ficheiro player.c que atualiza a posição do player. No caso de processamento de texto o scancode é enviado para o intercom que é um mecanismo de comunicação entre os dois jogadores através da função intercom_send_letter.

2.4. Funcionalidades do rato

O rato também funciona por interrupções e é utilizado na navegação nos menus e no controlo dos movimentos do jogador. As suas interrupções capturadas no `main_event_loop` chamam o handler (`mouse_ih` no ficheiro `mouse.c`) e para a in-game logic dos movimentos do rato é chamada a função `state_mouse_event` do ficheiro `game_state.c`. A posição do rato nos ecrãs de menus é inicializada e atualizada por um cursor (funções `cursor_move` e `cursor_get_position` no ficheiro `cursor.c`) para que o premir dos botões nos menus possa ser processado corretamente.

2.5. Funcionalidades da placa de vídeo

A placa de vídeo é utilizada para desenhar os vários ecrãs do jogo. Isto é feito essencialmente através do ficheiro `graphics.c` que define uma série de funções para os diferentes ecrãs do jogo, como por exemplo, `graphics_draw_game` ou `graphics_draw_menu`. Estas funções comunicam diretamente com as funções da placa de vídeo, no ficheiro `video.c`. A inicialização da placa de vídeo (mapeamento da VRAM e inicialização do modo de vídeo) é feita pela função `vg_start` do ficheiro `video.c` que é invocada na função `driver_setup` do ficheiro `driver_handler.c` que inicializa todos os mecanismos das drivers. O modo de vídeo utilizado é o modo direto 0x115 de resolução 800x600 com 24 bits por pixel para as cores (ou seja 2^{24} cores). É utilizado ainda double buffering por cópia para melhorar a performance e suavidade do jogo. Este processo é feito pela função `swap_buffer` no ficheiro `video.c` que é invocada nas funções de draw do ficheiro `graphics.c`.

2.6. Funcionalidades do RTC

O RTC é utilizado para gerar uma interrupção por alarme quando o jogador fica sem tempo para terminar o jogo. Isto é feito através do módulo `clock` que guarda o tempo restante que o jogador tem para terminar. Quando o jogador começa é chamada a função `clock_start_timer` que programa um alarme no RTC (função `rtc_set_alarm` no ficheiro `rtc.c`). Se o jogador pausar, o alarme do RTC será desativado e o seu tempo restante será guardado pela função `clock_stop_timer`. Quando o jogador sai da pausa a função `clock_start_timer` lê a hora do RTC (função `rtc_get_time`) e marca um alarme após o tempo restante de jogo passar. A interrupção gerada é captada como as anteriores na função `main_event_loop` e chama a função `state_rtc_event` que envia o jogador para o estado de derrota.

2.7. Funcionalidades da porta-série

A porta-série é utilizada no último nível do jogo para que ambos os jogadores possam completar o jogo juntos. Como esta ocasião é curta e não tem qualquer outro tipo de concorrência, a porta-série funciona por polling. Os dois jogadores trocam scancodes de teclas entre si pelo

que o protocolo de comunicação consiste apenas em 8 bits por char, sem controlo de paridade e com 1 bit de paragem onde são enviados estes scancodes com uma frequência máxima de 115200 bits por segundo mas que acaba por ser limitada pelo premir das teclas do utilizador. Este mecanismo de diálogo é tratado pelo jogo pelo módulo intercom recebe e envia scancodes pela função `intercom_send_letter` que por sua vez chama as funções `uart_receive_data` e `uart_send_data` do ficheiro `serial_port.c`.

3. Estrutura do código

3.1. driver_handler

Este módulo serve como interface entre as drivers e o jogo. Possui três funções, `driver_setup` que subscreve a interrupções, programa o modo de vídeo e configura a porta-série, `driver_cleanup`, que dessubscreve a interrupções e retorna ao modo de texto, e `main_event_loop`, a função que captura os interrupts de todas as drivers usando o `driver_receive` e chama as funções apropriadas do jogo.

3.2. kbc

Este módulo consiste nas funções do teclado desenvolvidas nas aulas, ou seja, subscrever e dessubscrever a interrupções e ler o output buffer para receber scancodes.

3.3 mouse

Este módulo consiste nas funções do rato desenvolvidas nas aulas, ou seja, subscrever e dessubscrever a interrupções, ativar e desativar data reporting, ler do output buffer e montar packets do rato.

3.4. RTC

Este módulo consiste em funções para o funcionamento do RTC, nomeadamente, subscrever e dessubscrever a interrupções, ativar e desativar interrupções por alarme, processar interrupções e ler as horas atuais.

3.5. serial_port

Este módulo consiste em funções para o funcionamento da porta-série, nomeadamente, configurar o protocolo de comunicação e receber e enviar dados.

3.6. timer

Este módulo consiste em funções para o funcionamento do temporizador maioritariamente desenvolvidas nas aulas práticas, ou seja, subscrever, dessubscrever e processar interrupções, obter a frequência do temporizador e ler o status register.

3.7. utils

Este módulo contém as funções utilitárias elaboradas nas aulas práticas, `util_sys_inb`, `util_get_MSB` e `util_get_LSB`.

3.8. video

Este módulo contém as funções para o funcionamento da placa de vídeo muitas das quais elaboradas nas aulas práticas, tais como, configurar um modo de vídeo, mapear VRAM, desenhar pixels, linhas, retângulos e XPMs, assim como trocar de buffer por cópia.

3.9. clock

Este módulo controla o tempo restante que o jogador possui para completar o jogo antes de ser derrotado. Contém funções para começar, parar e recomeçar o tempo. Estabelece a conexão entre o RTC e o jogo.

3.10. cursor

Este módulo controla o cursor do jogo nos vários menus. Contém funções para movimentar o cursor e obter a sua posição atual.

3.11. game_state

Este módulo controla o fluxo geral do jogo. Serve de centro entre os eventos gerados pelos dispositivos e toda a lógica do jogo. Como tal, contém funções para processar os eventos dos vários dispositivos e chamar os outros módulos apropriados relativos ao jogo.

3.12. graphics

Este módulo serve como a biblioteca de gráficos do jogo. Contém diversas funções para desenhar os vários ecrãs e estabelece a ponte entre a placa de vídeo e o jogo.

3.13. intercom

Este módulo age como conector entre a porta-série e o jogo. Recebe os scancodes do outro jogador e utiliza-os para a passagem do nível final. Assim, contém funções para a receção e transmissão com a porta-série, assim como verificações que o nível foi ultrapassado.

3.14 map

Este módulo contém as informações relativas ao mapa atual do jogo. As suas funções consistem em obter ou alterar certos objetos no mapa, obter as dimensões do mapa e carregar novos mapas.

3.15. player

Este módulo gere as informações relativas ao jogador e é constituído por funções que permitem mover o jogador, obter a sua posição atual e verificar se este está a atravessar para o mapa seguinte.

3.16. Estruturas de dados pertinentes

Algumas das estruturas de dados que consideramos pertinentes foram:

- Player, uma struct que contém a posição do jogador.
- Point2D, uma struct que define um ponto no plano bidimensional.
- Grid, um array bidimensional que guarda os diferentes elementos do mapa.

3.17. Funções principais

O projeto tem como função principal a função `main_event_loop` do ficheiro `driver_handler.c`. Esta função como já referido acima, recebe as interrupções via `driver_receive` e chama as funções apropriadas da lógica de jogo, o seu diagrama de chamadas está representado abaixo.

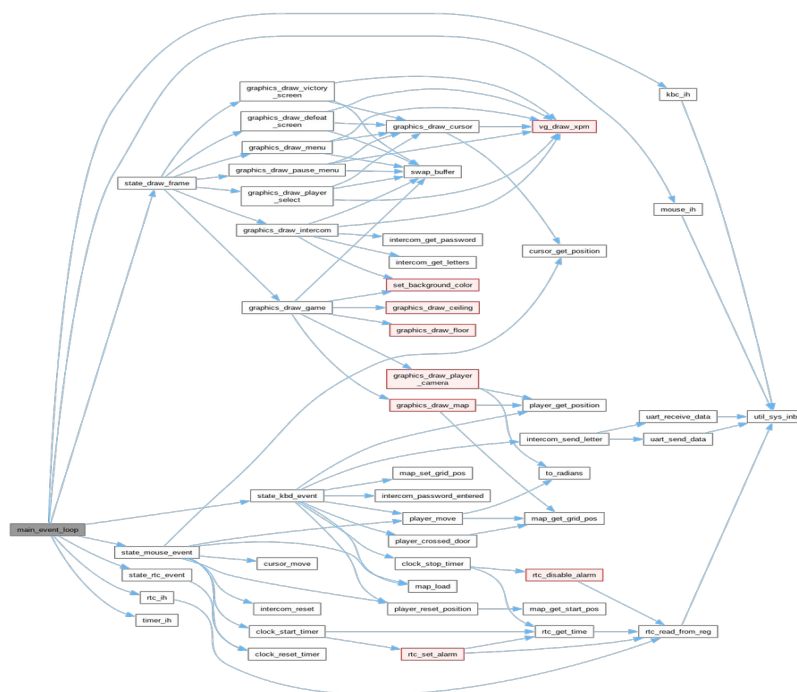


Figura 9. Diagrama de chamadas da função `main_event_loop`

4. Detalhes de implementação

4.1. Raycasting

A visão do jogador é feita por raycasting, usando o algoritmo de linhas de Bresenham. Estas linhas são coloridas usando aritmética modular a partir de texturas em XPMs de tamanho 16x16.

4.2. RTC start-and-stop

Para permitir que o jogador tivesse um limite temporal mas que pudesse pausar o jogo, foi necessário implementar um mecanismo que ativasse alarmes em função do tempo restante em vez de uma hora fixa. Para além disso, foi necessário implementar a possibilidade de desativar e reativar as interrupções por alarme.

4.3. Detecção de colisões

Para a navegação do jogador, foi obviamente necessário implementar a detecção de colisão com este com os objetos. Isto foi feito confirmando que sempre que o jogador se quer mover, a posição para onde ele irá está “desimpedida”.

4.4 Gestão do fluxo de jogo

Para a gestão do fluxo do jogo optámos pela construção de uma máquina de estados que é usada no módulo `game_state` de forma a garantir um event-driven design. Cada interação do utilizador é processada de acordo com o estado atual do jogo.

4.5. Modularização

De forma a garantir um desenvolvimento incremental, assim como a testabilidade do código já desenvolvido, estruturámos o código para funcionar de forma modular. As drivers são integradas no jogo através de módulos de conexão (por exemplo, o clock para o RTC) e as variáveis específicas de certos módulos são estáticas e apenas acessíveis para outros ficheiros através de funções.

5. Conclusões

5.1. Problemas do projeto

- O desenvolvimento do projeto foi parcialmente prejudicado pela redução de integrantes do nosso grupo na fase inicial.
- A existência de vários projetos de outras cadeiras em concorrência fez com que não pudessemos dedicar tanto tempo a este projeto como desejado.
- Apesar de ser propositada, a ausência de aulas práticas sobre o RTC e porta-série dificultaram a construção de código robusto para estes dispositivos.

5.2. Like-to-haves

- Com mais tempo gostaríamos de acrescentar uma melhor funcionalidade multi-jogador ao projeto.
- Teríamos feito também algumas melhorias à parte gráfica do projeto.
- Teríamos feito mais níveis para o jogo.

5.3. Principais conquistas do projeto

- Conseguimos desenvolver um código com uma boa estrutura que permitiria que continuássemos a desenvolver facilmente.
- Conseguimos implementar todas as funcionalidades desejadas.
- Conseguimos desenvolver bem o código de forma a que ficasse “orientado a objetos à moda de C”.

5.4. Lições aprendidas

- Aprendemos que devemos planejar bem as funcionalidades e definir bem as suas prioridades.
- Aprendemos que, mesmo com o relatório, usar uma abordagem incremental tem grandes benefícios.